Electrical Challenge: CAN Node Daughterboard

Name: Douglas Gray Applicant ID: 190

1. Rules

FSUK T11.7 Low Voltage Batteries

FSUK T11.7.12 “Must include overtemperature protection of at least 30 % of the cells, meeting EV5.8.4, that trips when any cell leaves the allowed temperature range according to the manufacturer’s datasheet, but not more than 60°C, for more than 1s and disconnects the battery,”, “It must be possible to display all cell voltages and measured temperatures, e.g. by connecting a laptop, Signals needed to fulfil these requirements are SCS, see T11.9.”

FSUK T11.9 System Critical Signal

FSUK T11.9.7 “The ESF must contain a detailed description of all the potential failure modes that can occur for each SCS, the strategy that is used to detect these failures and the tests that have been conducted to prove that the detection strategy works. The failures to be considered must include but are not limited to the failure of sensors and actuators, signals being out of range, corruption of the message and loss of messages and the associated time outs.”

FSUK T11.12 Sensors & Electrical Components Mounting

FSUK T11.12.1 “[AS T11.12.1 IN 2026 FORMULA STUDENT GERMANY RULES V1.1 WITH THE FOLLOWING ADDITION] For all mounts, T2.2.3 applies.”

FSG T2.2.3 Does not seem to exist? I can’t find it in the rules document, there is a T2.2.2 but then it goes directly into T2.3 after that? FSUK also does not have a T2.2.3 so I assume this is just a typo in the rules? FSG Section T2 (General Design Guidance) doesn’t have a particular section for electrical components as FSUK does, so I’ll assume it was meant to reference FSG T2.2, which is Ground clearance.

FSG T11.12 Sensors, Cameras & Electrical Components Mounting should cover this?

FSG T11.9 System Critical Signal (SCS)

FSG T11.9.2 “Any of the following SCS single failures must result in a safe state of all connected systems:

(a) Failures of signals transmitted by cable: • Open circuit • Short circuit to ground

(b) Failures of analog sensor signals transmitted by cable: • Short circuit to supply voltage

(c) Failures of sensor signals used in programmable devices: • Implausibility due to out of range signals, e.g. mechanically impossible angle of an angle sensor.

(d) Failures of digitally transmitted signals by cable or wireless: • Data corruption (e.g. checked by a checksum) • Loss and delay of messages (e.g. checked by transmission time outs)

Signals might be a member of multiple signal classes, e.g. analog signals transmitted by cable might be a member of T11.9.2.a, T11.9.2.b and T11.9.2.c."

FSG CV4.1 Shutdown Circuit

CV4.1.1 "The Shutdown Circuit (SDC) directly controls all electric power to the ignition, fuel injectors and all fuel pumps. It must act through a minimum of two mechanical relays."

Summary and Analysis of Rules

The rules I researched related to the restrictions with mounting sensors/electrical components, and how they must transfer data through the system for critical signals. As the task does not specify where this temperature sensor will be placed in the car, I also researched rules relating to possible locations that would require one.

FSUK T11.7 – Over-temp for longer than 1s must trip something to disconnect the battery, and the measured temperature must be able to be displayed by connecting a laptop. This would likely be handled by other systems rather than the sensor itself, but is worth noting.

FSUK & FSG T11.9 – System Critical Signal failures, which this temperature could be depending on the install location, have to be carefully considered for multiple ways they could fail, i.e. corruption, packet loss, physical sensor failure, etc. Many of these are likely handled on the data receiving side (the CAN) but some error handling from the daughterboard may be helpful.

FSG CV4.1 may be triggered by the above SCS, so I included a short excerpt from that section to have a reference for it.

2. Overview

My approach to the task was sequential, beginning with ensuring that I understood all the basic processes, components, and any problems or errors that I could face before moving on to any coding or schematic work. This helped me account for all scenarios in my code and have a greater holistic understanding of the systems at work, instead of overly focusing onto a small section and missing important steps.

My first step was to create a GitHub repo to store all my work and data within, allowing everything to be saved and tracked and helping me consider problems from multiple directions and within the context of the greater codebase.

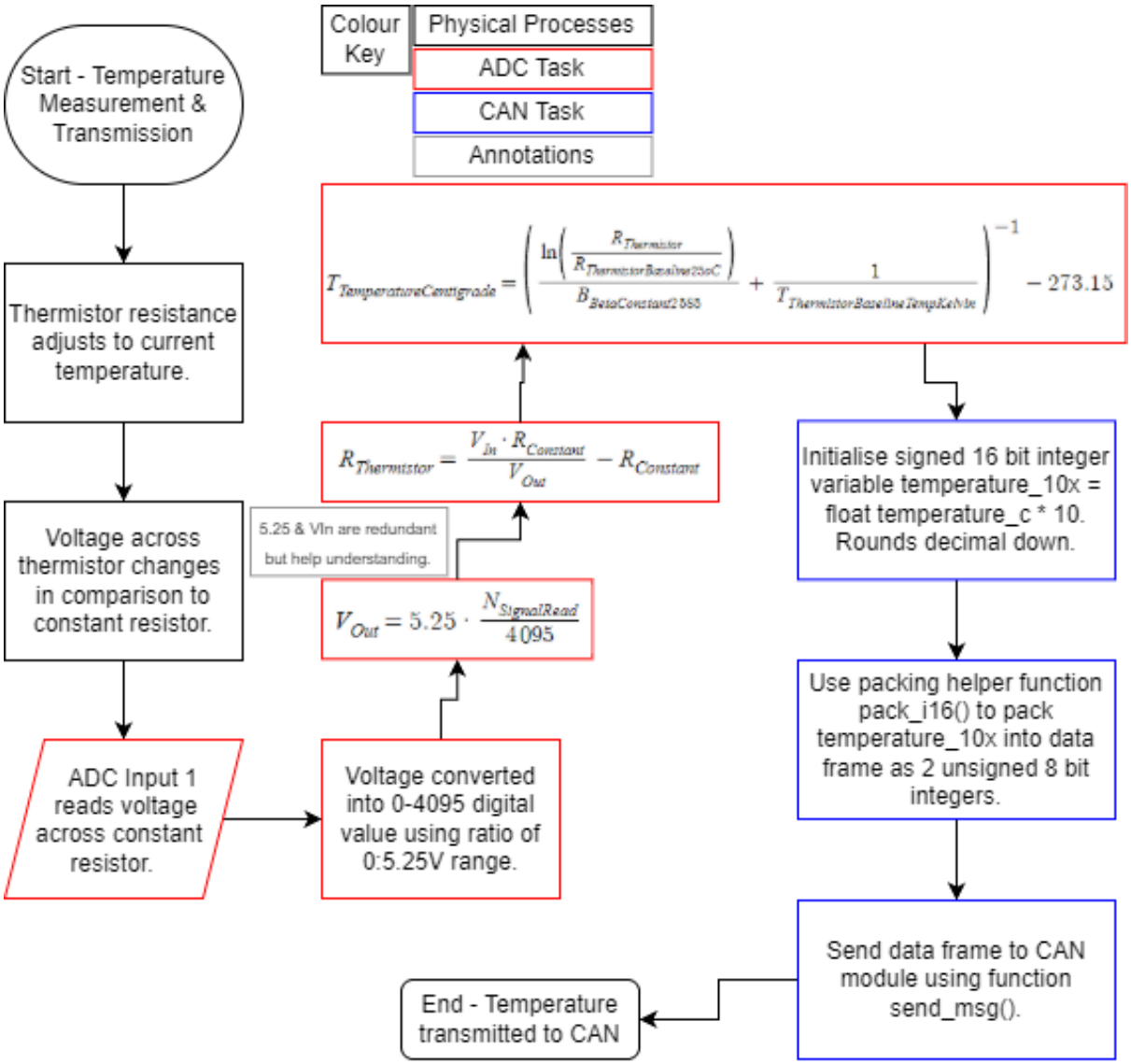
By creating a detailed flowchart and modelling the mathematical process using Desmos, I was able to ensure the ADC and CAN code was as lightweight as possible to ensure quick processing, and my researched knowledge of the components made the schematic creation process simpler as I understood the required connections.

Finally, I have made sure to look back at the end of the process so I can identify any further changes I can make to improve the system with a greater knowledge of the final structure.

GitHub Repo https://github.com/AerosmithTG/CAN-Node-Sensor-Interface-ID-190

3. Logic/block diagram

Flowchart



Physical Processes

These are what is physically happening on the circuit board, particularly across the voltage divider, as the temperature varies and the thermistor changes resistance.

ADC Task – Executed on Daughterboard

First the ADC completes its nominal purpose of converting the analogue voltage signal to a 12-bit digital signal, and then using voltage divider and an NTC beta constant equation, I solve for the temperature of the thermistor and return the function.

This currently states a voltage range up to 5.25V, but I moved to a 3.3V input to ensure compatibility with the microcontroller. This does not affect the code or the output as the ratio of voltage is what is used rather than the voltage value itself for the processing in ADC_task.cpp.

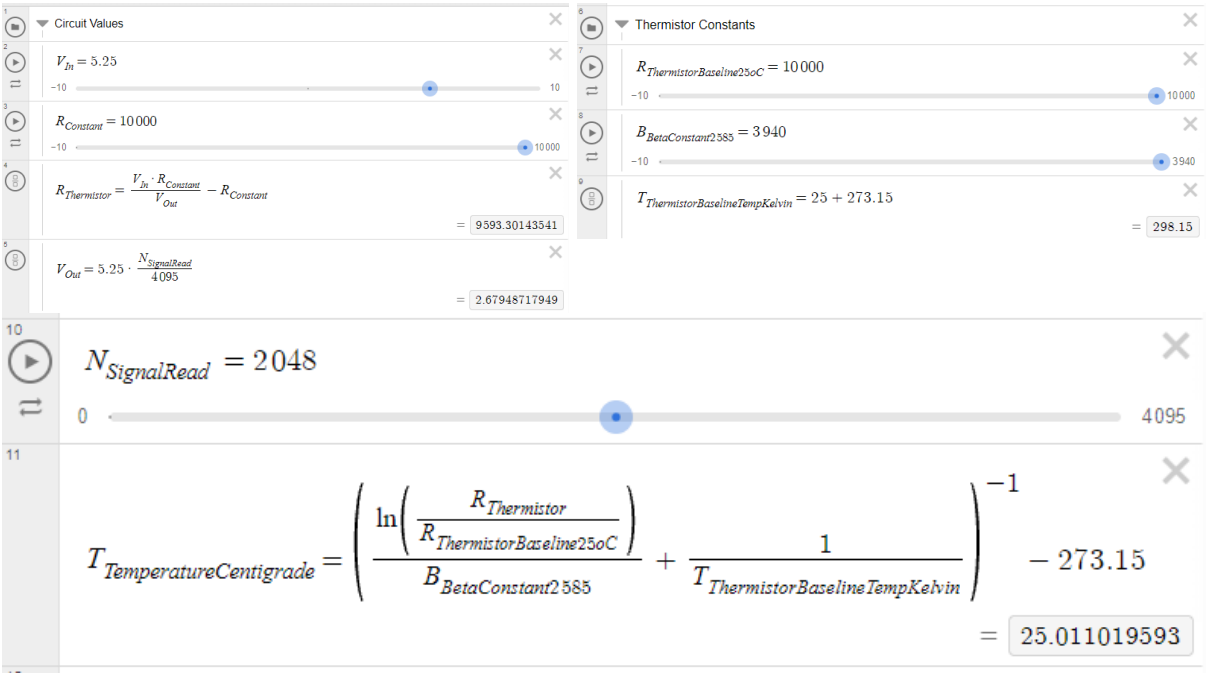
CAN Task – Executed on CAN Data Node

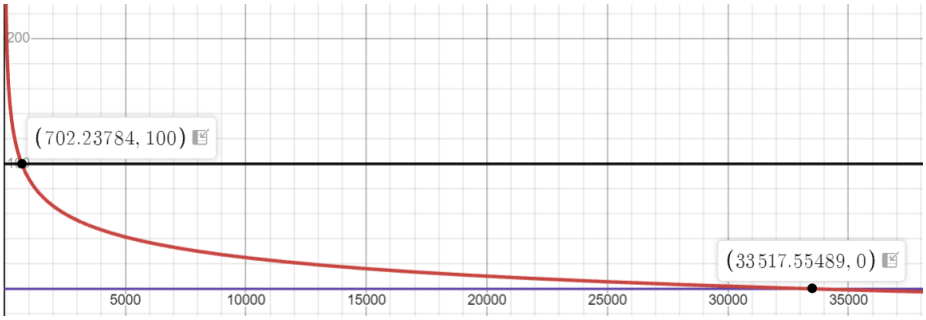
First, I convert the float value of temperature into a signed 16-bit integer value, multiplied by 10 to bring along the first decimal place. This value is then packed into the provided data frame using the packing helper function, which splits the signed 16-bit integer into two unsigned 8-bit integers (2 bytes of data). Finally, it is sent using the MSM_CAN::send_msg() function to the CAN data bus.

Mathematical Modelling

As mentioned in the overview, I have also created a Desmos Graph to model the inputs, constants, and 'outputs' for the ADC task processes, helping me to ensure the accuracy of the maths used.

https://www.desmos.com/calculator/gdqgglfel8

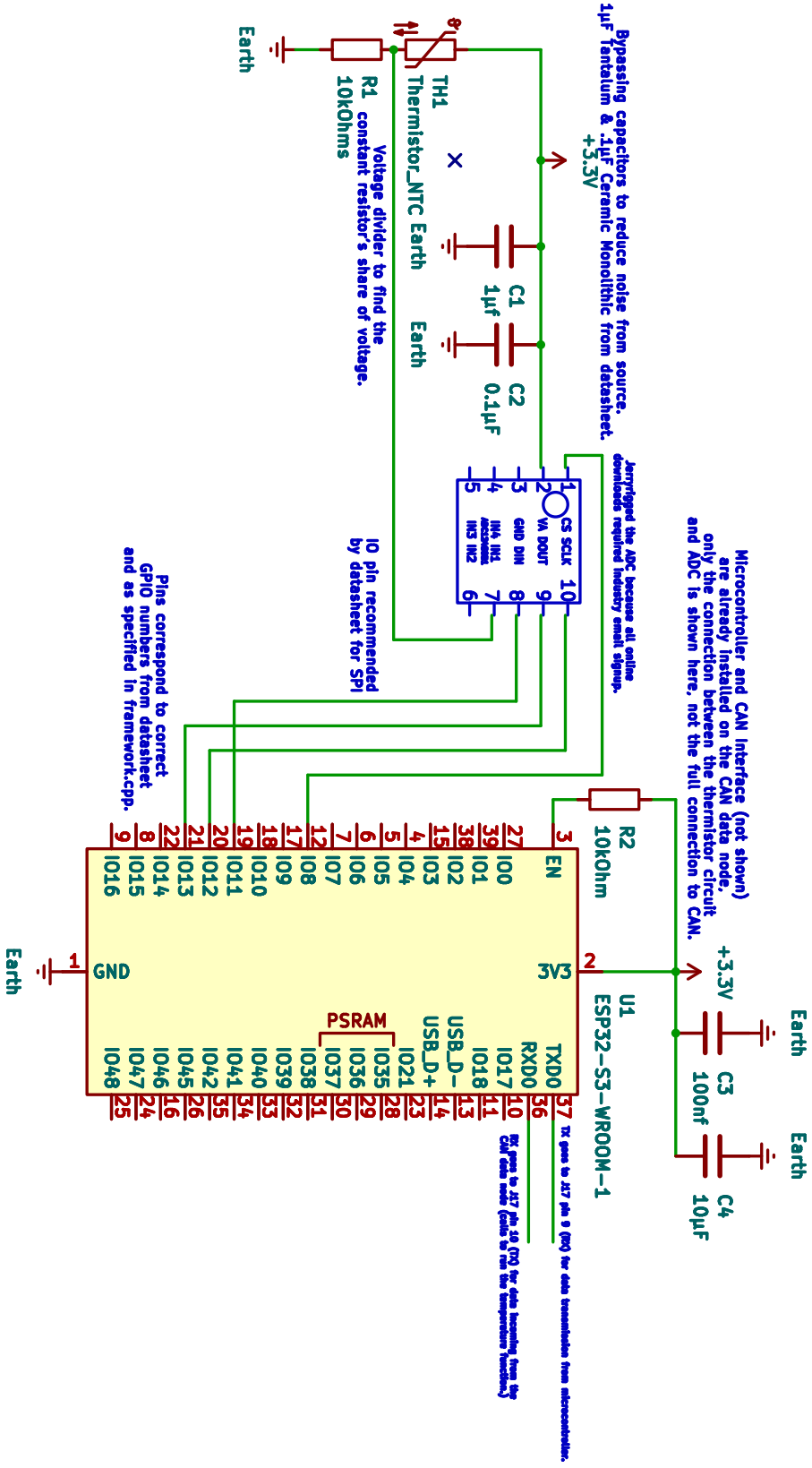




A graph of temperature against resistance for the specified thermistor. Points are the edge values for the range 0-100°C.

4. Schematic Design

KiCad Schematic with Annotations



The schematic includes 3 primary elements as follows:

Voltage Divider

A simple voltage divider to find the voltage across the 10kOhm constant resistor against the variable resistance of the NTC thermistor. The V_{out} from this goes to the input pin of the ADC to be converted into a digital signal.

Analogue Digital Converter

The ADC takes input on only 1 pin of the voltage across the constant resistor and translates it into a 12 bit signal. It is powered in parallel to the voltage divider from a 3.3V source, within it's operating range, with the recommended capacitors from the datasheet in place performing power bypassing to ensure any noise from the power source does not interfere.

The chip select, clock and data output and input pins are all connected to the microcontroller to the correct GPIO pins as specified in framework.cpp from the template code and the microcontrollers datasheet. This ensures that it stays in sync and can be called to with the read_adc() function.

Microcontroller

Powered by a separate 3.3V power source, also using capacitors to reduce noise from the source, and with a 10kOhm pull up resistor to provide a high signal to the EN pin so that it turns on properly.

As stated in the brief, the microcontroller is already integrated into the CAN node with a CAN interface, so in order to keep the schematic clear I have only detailed the connections to the J17 expansion, which are the following:

TX/RX: Transmit and receive pins for data transfer. Correspond to their inverses on the microcontroller so that data can be sent for and received.

To ensure the functionality of all components in the schematic, I made sure to verify the working conditions for each component involved.

Component Values

I extracted the relevant component values from the data sheets of each component. From the thermistor, I needed the baseline resistance and temperature, which was 10kOhms and 25°C respectively, and the beta constant (measured over the range 25 to 85°C, which should be accurate for our measurement range of 0-100°C), which was 3940. This allows me to calculate the temperature of the thermistor using its resistance, along with the resistance of the constant resistor, which I choose to be a 10kOhm resistor to best compare against the thermistor.

For the ADC, I needed the voltage range, which was 0 – 5.25V, and the resolution of its measurement, which was 12 bits. I powered it with a source of 3.3V, within its range and compatible with the source powering the microcontroller. I also used the datasheet to ensure I connected all the correct pins to the microcontroller.

Finally, for the microcontroller I ensured it was powered with a source compatible with its range of 3.0 – 3.3V, and used the datasheet to ensure all pins, especially the input pin taking the ADC output, were compatible with the correct data transfer types (SPI).

5. Software

ADC Task

```
#include "challenge.h"
#include <cmath>

// Runs once a second. read_adc() gives you a number from 0 to 4095.
bool adc_read(float &temperature_c)
{
    const float raw_adc = read_adc();
    if (!std::isfinite(raw_adc)) return false;
    temperature_c = NAN;

    //Circuit & Thermistor Constants
    //RCon - Resistance of constant resistance comparator resistor in the
voltage divider against the thermistor.
    //RThe - Datasheet resistance of thermistor at a temperature of 25 degrees
celsius.
    //BetaCon - Datasheet beta constant of thermistor between temps of 25 to
85 degrees celsius, used to calculate temperature from resistance.
    //ThermistorBaselineTemp - Temperature in kelvin (required for temp
calculations) at which thermistor reaches its baseline resistance.
    const float RConstant = 10000.0, RThermistor25Baseline = 10000.0;
    const float BetaConstant2585 = 3940.0;
    const float ThermistorBaselineTempKelvin = 25.0 + 273.15;


    //float VOut = 5.25 * raw_adc / 4095.0, simplified steps from flowchart
diagram but line here for clarity.
    float RThermistor = RConstant * 4095 / raw_adc - RConstant;
    //Temperature calculated using Beta Parameter Model (sufficient for this
application over Steinhart-Hart model as lacking required measurements and
level of precision not required.)
    //cmath power and natural log used, optimised math performance by
performing minimal amount of resource intensive exponentiation and logarithms.
    temperature_c = pow(((log(RThermistor / RThermistor25Baseline) /
BetaConstant2585) + pow(ThermistorBaselineTempKelvin, -1)), -1) - 273.15;

    //Maths code tested seperately outputs expected value for different
raw_adcs.

    return std::isfinite(temperature_c);
}

//File size is 2KB, should fit easily within Microcontroller's ROM capacity of
384KB.
```

CAN Task

```
#include "challenge.h"
#include "MSM_CAN.hpp"
#include <cmath>

void can_send_temperature(float temperature_c)
{
    MSM_CAN::TxFrame frame{};
    frame.id = 0x600;
    frame.length = 2;

    //Error checking to ensure results out of range are not sent
    if ((temperature_c < 0.0) || (temperature_c > 100.0)) {
        //Code to send an SCS error here
    }

    //integer 10x value of temperature_c, will always round down at 1dp (25.19
-> 251), level of precision is acceptable.
    int16_t temperature_10x = temperature_c * 10;
    //Convert to 16 bit signed, split into 2 bytes for payload prep. Frame
uses unsigned int, convert signed directly and CAN will change it back?

    //Packing helper for int16_t -> uint8_t (from MSM_CAN.hpp):
    //inline void pack_i16(uint8_t data[8], uint8_t index, int16_t value)
    //{
    //    pack_u16(data, index, static_cast<uint16_t>(value));
    //}
    MSM_CAN::pack_i16(frame.data, 0, temperature_10x);
    //Packing from index 0 to ensure both bytes of data are included.
    MSM_CAN::send_msg(frame);
    //Schedule unecessary as framework calls function per ADC sample.
}
```

Worked Example

raw_adc = 2500

RThermistor = 10000 * 4095 / 2500 – 10000 = 6380Ω

temperature_c = $((\ln(6380/10000) / (3940) + (25 + 273.15)^{-1})^{-1} – 273.15 = 35.4966$

temperature_10x = 354 (decimal) = 0162 (hexadecimal)

frame[0] = 01, frame[1] = 62

I also tested for RThermistor = 702 and 33518Ω (boundary values) and they returned the correct values of 100 and 0°C respectively.

6. Implementation

Spacing and Placement

The circuit itself is relatively simple, with the bulk of connections being pin to pin, but it would have to interface with the existing CAN system, so a mount that places it nearby to the data node would be necessary.

Certain features in the circuitry, such as the power decoupling capacitors must be placed in specific locations, in their case as close as possible to the power connection on the ADC and microcontroller (within 1 cm) for optimal working conditions. The grounding points across the system should all also go to the same ground to prevent voltage mismatch, so you would have to consider the placement with regards to proximity to one of the car's grounding wires.

Making

The component could be made with a very simple PCB design, or even direct wiring as there are only a few connections. Some complexity could be created because connections cannot cross over as they do in the schematic. Power would likely have to be drawn from an existing power circuit in the car, so I would have to ensure it was at the correct voltage of 3.3V, perhaps using a transformer, and that it would not be overly high current so it doesn't fry the components.

Testing

The code files are small enough to easily fit within the memory of the microcontroller, but rigorous testing of the system should be done before making any permanent changes to ensure everything is working properly before it is in a place where it is more difficult to access or manipulate. This could be done using the MSM GitHub repo loaded into a venv for software testing, and then with the actual components once software testing is complete.

To ensure the thermistor is working properly you could use a refrigerated or heated enclosed environment at a known temperature (using a thermometer) and compare the calculated value from the system to the true temperature.

The ADC's operational range is -40 – 85°C, so you should be careful not to overheat it during testing, and it may require some heat shielding to ensure it stays within this range during operation. You may also have to purchase the 105°C rated version of the microcontroller.

In addition to the testing within the range of 0-100°C, it would also be prudent to do testing outside of the range to ensure that the system properly recognises and reports an issue when this happens.

The CAN message can be checked using one of the unpacking helper functions from MSM_CAN.hpp to merge the uint8s from the data frame back into an int16 and checking if this matches the calculated value from the earlier stage. As our temperature range does not go into the negatives, there should be no difference between the unsigned and signed, but this should be properly verified to prevent any errors in the translation. This data should be able to be displayed using a laptop linked to the CAN system.

Any issues encountered during this should be narrowed down to a singular component/connection by progressive testing of each part of the system.

Thank you for your consideration!



